

TOPIC 01: WHAT IS JAVASCRIPT?

Definition:

JavaScript ek **high-level, dynamically typed** programming language hai jiska use websites ko **Interactive aur Dynamic** banane ke liye kiya jata hai.

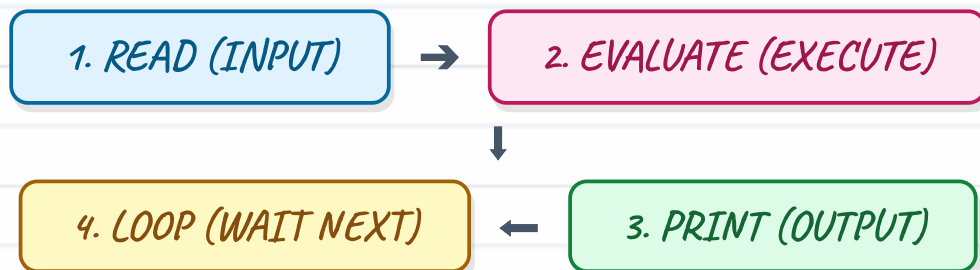


Exam Tip: JS is client-side by default in browsers, but Node.js allows running it on servers too!

TOPIC 02: CONSOLE & REPL

Console Workflow:

Browser Console **REPL** architecture par kaam karta hai:



TOPIC 03 & 04: VARIABLES & DATA TYPES

Variable: Name of a memory storage location.

JS is **Dynamically Typed** (variable ka type pehle se declare nahi karna padta).

7 Primitive Data Types:

- **Number** : 5, 3.14, -10
- **Boolean** : true, false
- **String** : "Tony Stark", 'Hello'
- **Undefined** : Variable declared but no value assigned
- **Null** : Intentional absence of any object value
- **BigInt** : Very large numbers ($> 2^{53} - 1$)
- **Symbol** : Unique immutable identifier

```
let a = 5; console.log(typeof a); // Output: "number"
```

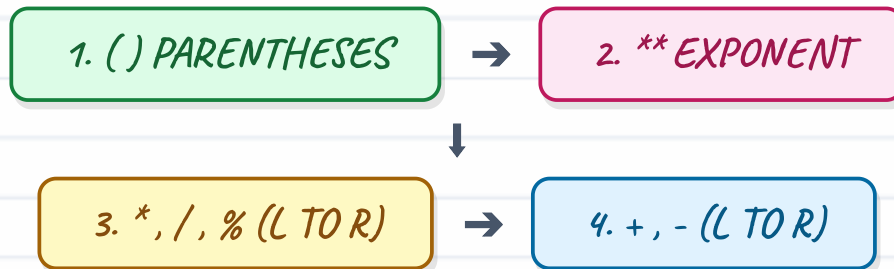


Topper Catch: `typeof null` returns **"object"** (famous historical bug in JS)!

TOPIC 05 & 06: OPERATIONS & PRECEDENCE

Arithmetic Operators: +, -, *, /, % (Modulo), ** (Power)

Operator Precedence Order (High → Low):



TOPIC 07 & 08: ASSIGNMENT & UNARY OPERATORS

Assignment: =, +=, -=, *=, %=

Unary Operators:

Operator	Name	How it works	Example
a++	Post-Increment	Use old value first, then add 1	let a=5; b=a++; // b=5, a=6
++a	Pre-Increment	Add 1 first, then use new value	let a=5; b=++a; // b=6, a=6
a--	Post-Decrement	Use old value first, then minus 1	let a=5; b=a--; // b=5, a=4
--a	Pre-Decrement	Minus 1 first, then use new value	let a=5; b=--a; // b=4, a=4

TOPIC 09 TO 12: NAN, IDENTIFIERS & CASING

- **Topic 09 (NaN):** Not-a-Number invalid result (e.g. $0/0 = \text{NaN}$). Note: `typeof NaN === "number"`.
- **Topic 10 (Identifier Rules):** Names can have letters, digits, `_`, `$`. First char **cannot** be a number!
- **Topic 11 (Casing):** `camelCase` (`userFirstName`), `PascalCase` (`UserProfile`), `snake_case` (`user_name`).
- **Topic 12 (TypeScript):** Static-type superset of JS. Types compile time pe check hoti hain.

TOPIC 13 & 14: STRING INDICES & CONCATENATION

String Indexing (0-based):

```
let name = "Tony"; console.log(name[0]); // Output: 'T'
console.log(name[3]); // Output: 'y' console.log(name.length); // 4
```

Concatenation (+):

```
let s = "Tony" + " " + "Stark"; // "Tony Stark"
```

TOPIC 15: NULL VS UNDEFINED

Null	Undefined
Developer intentionally sets value as empty: let a = null;	Variable declared but value not yet assigned: let a;
typeof a === "object"	typeof a === "undefined"

TOPIC 16 & 17: SCRIPT LINKING & TEMPLATE LITERALS

Script Tag: `<script src="file.js"></script>` (Link at bottom of body).

Template Literals (Backticks):

```
let a = 5; let msg = `Number is ${a}`; // Output: "Number is 5"
```



Exam Tip: Template literals allow embedded expressions and multi-line strings easily without `\n`!

TOPIC 18: COMPARISON OPERATORS

Loose vs Strict Equality:

- `==` (Loose): Checks value only (type coercion karta hai). `5 == "5"` is **true**.
- `===` (Strict): Checks **Value + Data Type**. `5 === "5"` is **false!** (Always Recommended )
- **Unicode Order**: `'a' > 'A'` is **true** (small 'a' code=97, 'A' code=65).

TOPIC 19, 20, 21: CONDITIONALS, SWITCH & LOGIC

Conditionals: `if (...) { } else if (...) { } else { }`

Switch Statement: Compares expression against case values with mandatory `break`;

Logical Operators: `&&` (AND), `||` (OR), `!` (NOT).

TOPIC 22: TRUTHY & FALSY VALUES

Falsy Values (Total 7 Fixed Values):

1. **FALSE**

2. **0, -0**

3. **0N (BIGINT)**

4. **"" (EMPTY)**

5. **NULL**

6. **UNDEFINED**

7. **NAN**

Truthy Values: In 7 ko chhodkar baaki sab truthy! (e.g. `" "`, `"false"`, `[]`, `{}`, non-zero numbers).

TOPIC 23: ALERT() AND PROMPT()

`alert("Hello")`: Shows pop-up dialog.

`prompt("Enter name:")`: Takes input from user (returns String).

TOPIC 24 & 25: STRING METHODS

Immutability: Strings cannot be changed in-place! Methods always return a new string.

Method	Action	Example	Output
<code>trim()</code>	Removes spaces from start & end	<code>" hello ".trim()</code>	<code>"hello"</code>
<code>toUpperCase()</code>	Converts to uppercase	<code>"hello".toUpperCase()</code>	<code>"HELLO"</code>
<code>toLowerCase()</code>	Converts to lowercase	<code>"HELLO".toLowerCase()</code>	<code>"hello"</code>
<code>indexOf(str)</code>	First matching index (-1 if not found)	<code>"hello".indexOf("l")</code>	<code>2</code>
<code>slice(start, end)</code>	Cuts piece from start to end-1	<code>"hello".slice(1, 4)</code>	<code>"ell"</code>
<code>replace(old, new)</code>	Replaces first occurrence	<code>"hello".replace("l", "o")</code>	<code>"heolo"</code>
<code>repeat(n)</code>	Repeats string n times	<code>"hi".repeat(3)</code>	<code>"hihihi"</code>

Method Chaining:

```
let str = " hello "; let res = str.toUpperCase().trim(); // "HELLO"
```

TOPIC 26: ARRAYS (LINEAR DATA STRUCTURE)

Arrays are **Mutable** (can be modified by index directly: `arr[0] = 99`).

`UNSHIFT(VAL) → [START]`

`[END] → PUSH(VAL)`

`SHIFT() ← [START]`

`[END] ← POP()`

Other Methods: `concat()`, `reverse()`, `includes()`, `indexOf()`, `slice()`, `sort()`.
`splice(start, deleteCount, item1...)`: In-place item add, remove ya replace karta hai.

TOPIC 27: ARRAY REFERENCE IN MEMORY

Address Pointing: Array variable actual data nahi balki memory reference store karta hai!

`LET ORIGINAL = [1, 2, 3]`

→ Points to:

`ADDRESS: 0X01 [1, 2, 3]`

`LET COPY = ORIGINAL`

→ Points to:

`SAME ADDRESS: 0X01`

Isliye `copy.push(4)` karne par original array bhi change ho jata hai!

TOPIC 28 & 29: CONST ARRAY & LOOPS

Topic 28: `const arr = [1, 2]`; arr reference locked hai, par elements mutate ho sakte hain.

Topic 29: Loops: `for`, `while`, and `for...of` (iterates directly over array values: `for(let fruit of fruits)`).

TOPIC 30, 31, 32: OBJECTS IN JAVASCRIPT

Object Literals (key-value pairs):

```
const student = { name: "Kamal", age: 23 }; // Access: student.name OR student["age"] // Add/Update: student.city = "Delhi"; // Delete: delete student.age;
```

JS Auto-Conversion: Object keys hamesha automatically string ban jati hain.

Topic 31: Nested Object: classInfo = { aman: { grade: "A+" } };

Topic 32: Array of Objects: classInfo = [{ name: "Aman" }, { name: "Karan" }];

TOPIC 33: MATH OBJECT

Built-in Math Properties & Methods:

- Math.PI : 3.14159... | Math.E : 2.718...
- Math.abs(n) : Returns absolute positive value (-5 → 5).
- Math.pow(a, b) : Power calculation (a^b).
- Math.floor(n) : Rounds down to nearest integer (3.9 → 3).
- Math.ceil(n) : Rounds up to nearest integer (3.1 → 4).
- Math.random() : Generates decimal number between 0 and 1.



Formula for Random Integer 1 to 10: `Math.floor(Math.random() * 10) + 1`

TOPIC 34 & 35: FUNCTIONS IN JS

```
function sum(a, b) { return a + b; } let ans = sum(5, 3); // Output: 8
```

TOPIC 36: SCOPE IN JAVASCRIPT

1. FUNCTION SCOPE (VAR)

2. BLOCK SCOPE (LET, CONST)

3. Lexical Scope:

Inner function apne outer (parent) function ke variables ko access kar sakta hai! Lekin outer function inner ke variable ko access nahi kar sakta.

TOPIC 37 TO 40: HOF, METHODS & 'THIS'

- Topic 37 (Function Expression): `const sum = function(a, b) { return a+b; };`
- Topic 38 (Higher Order Function): Function jo doosre function ko argument leta hai ya return karta hai.
- Topic 39 (Methods): Object ke andar function property. Shorthand: `add(a, b) { return a + b; }.`
- Topic 40 ('this' Keyword): Points to the current object executing the function!

TOPIC 41: TRY AND CATCH

Error anticipate karne ke liye use hota hai taaki script ka execution ruk na jaye:

```
try { console.log(x); // x is not defined } catch (err) {  
  console.log("Error handled:", err); }
```

TOPIC 42 TO 45: ARROW FUNCTIONS & TIMERS

Arrow Function Syntax:

`const add = (a, b) => a + b;` (Clean, modern syntax, implicit return).

Timers:

`setTimeout(func, 2000)` : Run once after 2 seconds.

`let id = setInterval(func, 1000); clearInterval(id);` : Run repeatedly.



'this' Difference: Normal function ka 'this' calling object hota hai, jabki Arrow function ka apna 'this' nahi hota (lexical parent scope se inherit karta hai)!

TOPIC 46: MODERN ARRAY METHODS

Method	Behavior	Return Value
forEach()	Har element pe action karta hai (DOM/log)	Always undefined (No chaining)
map()	Return value se naya array banata hai	New Array (Chaining )
filter()	Condition true wale elements rakhta hai	Filtered Array
every()	Logical AND: Sabhi true ho to true	Boolean (true/false)
some()	Logical OR: Koi ek bhi true ho to true	Boolean (true/false)
reduce()	Pure array ko 1 value me condense karta hai	Single Accumulated Value

TOPIC 47 TO 51: SPREAD, REST & DESTRUCTURING

- **Spread Operator (...arr):** Elements ko expand/faila deta hai (Math.min(...arr) ya deep copy let copy = [...arr]).
- **Rest Parameter (...args):** Unlimited function arguments ko single array me collect karta hai. Must be last parameter!
- **Destructuring:**
Array: let [a, b] = [10, 20];
Object: let { name, age } = student;

TOPIC 52 & 53: DOM & ELEMENT SELECTION

DOM (Document Object Model):

Logical tree of webpage elements. Inspect via
`console.dir(document).`

DOCUMENT → <HTML> → <BODY> → <DIV>...

Selection Methods:

- `document.getElementById("my-id")` → Single element.
- `document.getElementsByClassName("box")` → `HTMLCollection`.
- `document.querySelector(".box p")` → First matching element.
- `document.querySelectorAll("p")` → All matching (`NodeList`).

TOPIC 54 TO 57: CONTENT, STYLE & DOM MODIFICATION

Text: `innerText` (visible text), `textContent` (raw text), `innerHTML` (full HTML tags).

Classes: `classList.add()`, `remove()`, `contains()`, `toggle()`.

Navigation: `parentElement`, `children`, `previousElementSibling`,
`nextElementSibling`.

Add/Delete: `document.createElement('p')` → `appendChild()` / `prepend()` →

Delete via `el.remove()`.

TOPIC 58 TO 65: DOM EVENTS & EVENT DELEGATION

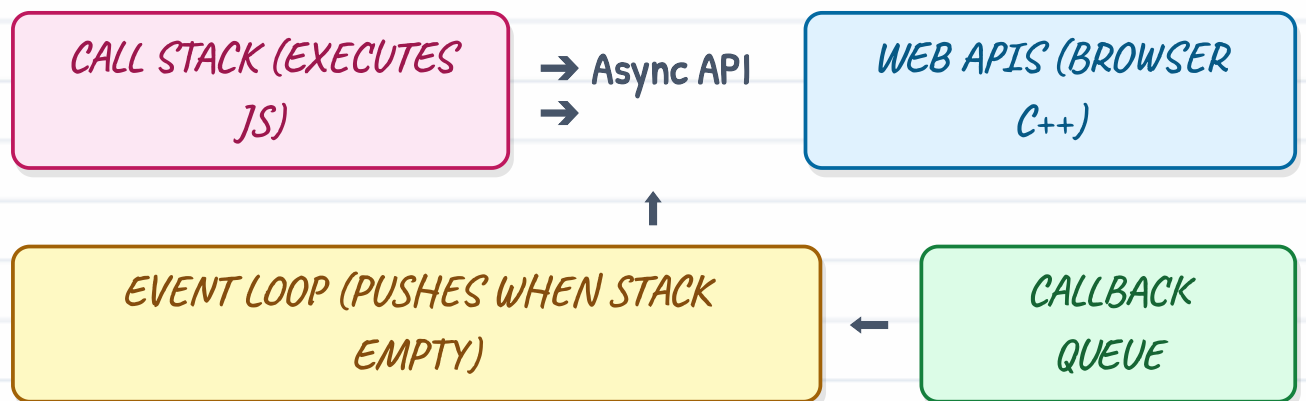
addEventListener:

```
btn.addEventListener("click", function() { // 'this' points to btn  
  element this.style.backgroundColor = "blue"; });
```

- `event.preventDefault()`: Form submit pe page refresh hone se rokta hai.
- `change` (focus out pe fire hota hai) vs `input` (real-time typing pe instant fire hota hai).
- **Event Bubbling**: Event child se parent ki taraf upar bubble karta hai (Stop via `event.stopPropagation()`).
- **Event Delegation**: Dynamic elements ke liye listener ko parent par attach karna.

TOPIC 66 TO 68: EVENT LOOP & ASYNC ARCHITECTURE

JavaScript is **Single Threaded** (sirf 1 Call Stack). Non-blocking kaise banta hai?



TOPIC 69 TO 73: PROMISES & PROMISE CHAINING

Topic 69 (Callback Hell): Nested callbacks (Pyramid of Doom). Solution → Promises!

Promise States:

PENDING (IN PROGRESS)

✓ Success ✗ Failure

FULFILLED (RESOLVE) →
.THEN()

REJECTED (REJECT) →
.CATCH()

```
saveToDB("Data 1") .then(res => saveToDB("Data 2")) // Chaining  
  .then(res => console.log("All Saved")) .catch(err => console.log("Common  
Catch:", err));
```

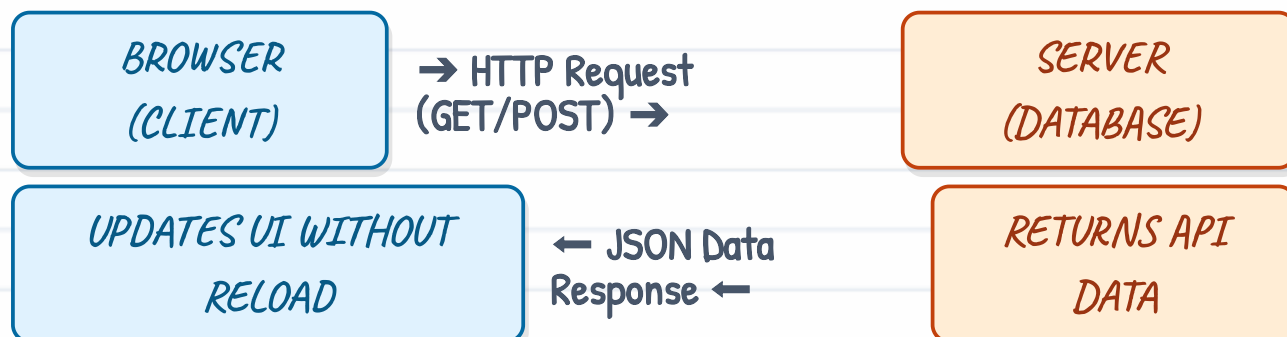
TOPIC 74 TO 76: ASYNC & AWAIT

- async function: Automatically returns a Promise!
- await: Pauses execution until Promise settles (Can only be used inside async).
- Error handling: Standard try...catch block use hota hai.

```
async function loadData() { try { let res = await saveToDB("Apna Data");  
  console.log(res); } catch (err) { console.log("Caught error:", err); } }
```

TOPIC 77 TO 81: APIS, JSON & HTTP STATUS

Client-Server Request Cycle:



JSON Methods: `JSON.parse(str)` (JSON to JS Object), `JSON.stringify(obj)` (JS Object to JSON String).

HTTP Status: 200 (OK), 400 (Bad Request), 404 (Not Found), 500 (Server Error).

TOPIC 82 TO 88: FETCH API VS AXIOS

Feature	<code>fetch()</code> API	Axios Library
Parsing	2-step: <code>res = await fetch()</code> , then <code>data = await res.json()</code>	1-step: Direct parsed in <code>res.data</code> ✅
Installation	Built-in in browser	CDN script / npm package

Axios Implementation:

```
async function getFact() { try { const config = { params: { country: "India" } }; let res = await axios.get("https://api.example.com/data", config); console.log(res.data); } catch (err) { console.log(err); } }
```



Summary: All 88 topics covered vertically in pure Topper Handwritten Notes style! Ready for print & revision!